Eine Lampe für Verliebte

In unserem Projekt „LoversLamp“ synchronisieren sich mehrere ESP-gesteuerte Lampen übers Internet per MQTT miteinander, um einer lieben, aber entfernten Person zu zeigen, dass man an sie denkt.

von Daniel Schwabe

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

Mittlerweile ist es klar: Die Make-Redaktion besteht aus unverbesserlichen Romantikern. Mit dem Flirt-Amulett in Ausgabe 7/25, Seite 34, und mit dem Herzschrittnehmer in dieser Ausgabe auf Seite 90 ist der Fall eindeutig: Wir lieben nicht nur Technik, sondern auch unsere Mitmenschen.

Für all die Leute, die über unsere Flirt-Amulette ihre bessere Hälfte gefunden haben, aber noch nicht zusammengezogen sind oder in einer Liebes- oder Freundesbeziehung über große Entfernung leben, haben wir die DIY-LoversLamp entwickelt

Mit dieser Lampe könnt ihr jederzeit sehen, ob eure Liebsten an euch denken. Wenn eine Person ihre Lampe einschaltet, geht die andere synchron an – egal, wo auf der Welt sie sich befindet.

Die Idee hinter der LoversLamp

Die LoversLamp ist eine smarte Lampe, die mit anderen identischen Lampen über das Internet verbunden ist. Wenn eine Person ihre Lampe einschaltet, wird diese Information an

Kurzinfo

» Synchronisierung über MQTT
» Steuerung über Touchfunktion des ESP32
» Verbindung zum Internet über TLS/SSL

Checkliste

Zeitaufwand:
3 Stunden

Kosten:
60 Euro

Material

» **ESP32**
» **LED-Streifen** Typ WS2812b
» **Plastikrohr** Durchmesser 27 mm
» **Lampenfuß**
» **Lampenschirm**
» **Kabel**

Werkzeug

» **Dremel**
» **Rohrzange**

Mehr zum Thema

» Dr. Stefan Recksiegel, Flirt-Amulette mit ESP-NOW, Make 7/24, S. 34
» Thomas Garaio, Weihnachtsstern mit Oxocard, Make 7/24, S. 50

Alles zum Artikel im Web unter *make-magazin.de/xyf6*

MQTT

MQTT ist ein besonders leichtgewichtiges Kommunikationsprotokoll, das speziell für den Datenaustausch zwischen Geräten mit geringer Leistung und instabilen Netzwerken bzw. Netzwerken mit geringer Bandbreite entwickelt wurde. Es basiert auf dem Prinzip des Publish-Subscribe-Systems. Geräte können Nachrichten zu einer bestimmten Topic (Überschrift) auf einem Server veröffentlichen und auch bestimmte Topics abonnieren, um direkt die neuesten Nachrichten zu diesem Thema angezeigt zu bekommen. Durch seinen geringen Ressourcenverbrauch eignet sich MQTT ideal für Anwendungen im Internet der Dinge (IoT), bei denen Sensoren, Maschinen oder smarte Geräte miteinander kommunizieren müssen.

Je nachdem, wie wichtig sie sind, kann man gesendete MQTT-Nachrichten in drei Kategorien einteilen:

QoS 0 bis 1. Bei Level 0 wird eine Nachricht einmal gesendet und es findet keine Überprüfung statt, ob sie angekommen ist.

Stufe 1 überprüft den Eingang einer Nachricht beim Empfänger und sorgt dafür, dass sie bei Bedarf noch einmal gesendet wird, falls der Empfänger nicht antwortet.

Und Stufe 2 stellt sicher, dass die Nachricht genau einmal ankommt, indem es einen Handshake zwischen Sender und Empfänger gibt. Im Gegensatz zu Level 1 kann hier eine Nachricht nicht zufällig mehrfach gesendet werden.

Das Publish-Subscribe-Prinzip, kurz Pub/Sub, ist ein Kommunikationsmodell, bei dem Sender und Empfänger nicht direkt miteinander verbunden sind, sondern über eine zentrale Instanz, den Broker (hier HiveMQ), Nachrichten austauschen. Dabei gibt es zwei Hauptakteure: den Publisher, der Nachrichten veröffentlicht, und den Subscriber, der diese Nachrichten empfängt. Ein Publisher sendet seine Nachrichten an ein bestimmtes Thema, die sogenannte Topic, während ein Subscriber sich für ein oder mehrere Topics anmelden kann, um entsprechende Nachrichten zu erhalten. Der Broker übernimmt die Vermittlung, indem er eingehende Nachrichten an alle Subscriber verteilt, die die entsprechende Topic abonniert haben.

Ein wesentlicher Vorteil dieses Modells ist die vollständige Entkopplung von Publishern und Subscribern. Der Publisher muss nicht wissen, wer die Nachrichten empfängt, und der Subscriber muss nicht wissen, von wem sie gesendet werden.

Durch diese Trennung kann man problemlos weitere Geräte in ein Netzwerk einbauen, das über MQTT kommuniziert.

Im Falle der Lampe funktioniert das Ganze wie folgt:

Die Logik der Lampe läuft in einem Loop: Erst wird überprüft, ob lokal geschaltet wurde, danach, ob online ein neuer Status hinterlegt wurde. Die Lampe überprüft „Wurde ich lokal ein-/ausgeschaltet?". Ist das der Fall, schaltet die Lampe das Licht um und sendet an den MQTTBroker (HiveMQ) die passende Nachricht „ON" oder „OFF"). Danach überprüft die Lampe „Wie ist der letzte Status auf HiveMQ?". Wenn sich der vom lokalen Status unterscheidet, schaltet die Lampe auf den aktuell online gestellten um. Die Überprüfung geschieht auch, wenn die Lampe gerade erst einen neuen Status gesendet hat. Aber da dieser natürlich mit dem lokalen Status übereinstimmt, schaltet die Lampe nicht um.

Keine der Lampen kommuniziert selbst mit einer anderen oder verwaltet den Status des Lampen-Netzwerks. Dadurch können beliebig viele LoversLamps miteinander verbunden werden. Diese können dann natürlich auch verschiedene Formen annehmen.

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

Wie wähle ich einen öffentlichen MQTT-Server aus?

Bei der Auswahl eines öffentlichen MQTT-Brokers gibt es einige wichtige Kriterien, die beachtet werden sollten. Ein MQTT-Server spielt eine zentrale Rolle in der Kommunikation bei IoT-Projekten. Bei der LoversLamp werden zwar keine Staatsgeheimnisse übertragen, aber man sollte direkt mit der Best Practice anfangen und immer den Sicherheitsaspekt im Hinterkopf haben.

1. Sicherheit und Verschlüsselung

Ein öffentlicher MQTT-Server muss die TLS-Verschlüsselung (Transport Layer Security) unterstützen, damit die Datenübertragung zwischen einer Lampe und dem Server sicher bleibt. Ohne TLS könnten Angreifer die Nachrichten abfangen, manipulieren oder sogar unerwünschte Befehle an verbundene Geräte senden. Ein guter Broker bietet standardmäßig eine verschlüsselte Verbindung über Port 8883 an und stellt ein gültiges Root-Zertifikat zur Verfügung, mit dem sich der ESP32 sicher mit dem Server verbinden kann. Wie genau TLS funktioniert und was es mit diesem Root-Zertifikat auf sich hat, steht im Kasten TLS.

2. Datenschutz und Serverstandort

Da IoT-Geräte Daten aus dem heimischen Smart Home übertragen, muss man darauf achten, dass der MQTT-Server in einem datenschutzkonformen Land gehostet wird. Server innerhalb der EU unterliegen der DSGVO (Datenschutz-Grundverordnung), was sie zu einer besseren Wahl macht als Server in Ländern wie den USA oder China. Der hier verwendete Dienst HiveMQ ist ein Anbieter aus Deutschland und somit eine gute Wahl.

3. Verfügbarkeit und Stabilität

Ein guter MQTT-Broker sollte hohe Verfügbarkeit haben, damit die damit verbundenen Geräte jederzeit zuverlässig funktionieren. Ein instabiler Server, der häufig ausfällt oder lange Latenzzeiten hat, würde zu Verzögerungen oder sogar Kommunikationsabbrüchen zwischen Geräten führen. Bevor man sich für einen Anbieter entscheidet, sollte man daher prüfen, ob es Erfahrungsberichte über die Verfügbarkeit gibt oder ob der Anbieter selbst eine Statusseite mit Live-Informationen zur Erreichbarkeit bereitstellt.

4. Kosten und Nutzungslimits

Viele öffentliche MQTT-Server bieten eine kostenlose Nutzung an, haben aber Einschränkungen in Bezug auf Nachrichtenfrequenz, Verbindungsanzahl oder Datenmenge. Vor der Wahl eines Brokers sollte überprüft werden, ob die kostenlosen Limits für das eigene Projekt ausreichen. HiveMQ bietet beispielsweise in seiner kostenlosen Cloud-Variante bis zu 100 gleichzeitige Geräteverbindungen und maximal 10.000 Nachrichten pro Monat, was für die LoversLamp völlig ausreichend ist.

5. MQTT-Protokollversion

Die meisten modernen MQTT-Server unterstützen MQTT 3.1.1 oder MQTT 5.0. Während MQTT 3.1.1 für die meisten Anwendungen ausreicht, bietet MQTT 5.0 einige erweiterte Funktionen wie verbesserte Sicherheitsmechanismen und eine effizientere Datenübertragung.

alle verknüpften Lampen gesendet, die dann ebenfalls angehen.

Technisch basiert jede Lampe auf einem ESP32-Mikrocontroller, der als Steuerungseinheit für die jeweilige Lampe dient. Die Kommunikation erfolgt über das MQTT-Protokoll, das speziell für die Signalübertragung in IoT-Netzwerken entwickelt wurde.

Warum ein öffentlicher MQTT-Server?

Damit die LoversLamp über das Internet funktioniert, benötigen wir einen zentralen Vermittler, einen sogenannten MQTT-Broker. Dieser empfängt die Nachrichten der Lampen und verteilt sie an alle verbundenen Geräte.

Eine Möglichkeit wäre, einen eigenen MQTT-Server zu betreiben. Das würde jedoch bedeuten, dass man eine eigene Infrastruktur bereitstellen und Ports in der heimischen Firewall öffnen müsste – ein potenzielles Sicherheitsrisiko. Um dies zu vermeiden, nutzen wir einen öffentlichen MQTT-Broker. Nach welchen Kriterien man einen öffentlichen Server auswählen sollte, steht im Kasten „Wie wähle ich einen öffentlichen MQTT-Server aus?".

In diesem Artikel wird HiveMQ verwendet. HiveMQ ist ein deutscher MQTT-Dienst, der eine kostenlose Cloud-Variante anbietet. Es ist eine zuverlässige und datenschutzkonforme Lösung, die perfekt für dieses Projekt geeignet ist. Mit HiveMQ kann man sicherstellen, dass die LoversLamp verlässlich und sicher funktioniert.

Anmeldung bei HiveMQ

Um HiveMQ nutzen zu können, muss man dort einen Account anlegen. Die Webseite dafür ist www.hivemq.cloud. Nach der Anmeldung muss man seinen „Nutzungsplan" auswählen. Der kostenlose ist perfekt für so ein kleines Projekt geeignet. Man bekommt damit 100 Verbindungen, 10 GB Datenaustausch pro Monat und, ganz, ganz wichtig, MQTT über TLS, also mit Verschlüsselung.

Hat man sich angemeldet, bekommt man einen „Cluster Overview" angezeigt. Hier findet man sowohl die personalisierten Server-URLs als auch die Ports, um MQTT zu verwenden. Beides muss später im Sourcecode nachgetragen werden.

Bevor man das allerdings macht, legt man noch Zugangsdaten für die LoversLamp-Clients an.

Dafür scrollt man auf der Übersichtsseite nach unten und öffnet das „Access Management". Dort klickt man auf den großen gelben „Add new Credentials"-Button und wählt einen Usernamen, ein Passwort und setzt die Permissions auf „Publish and Subscribe". Als Letztes dann auf den gelben „Save"-Button klicken.

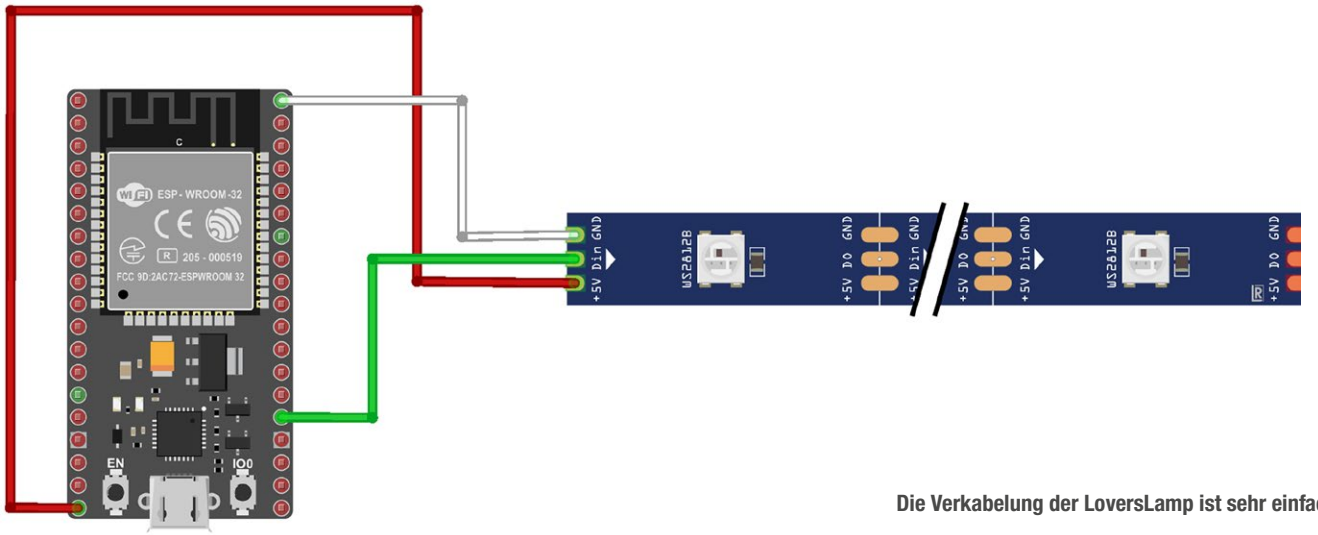
Sicherheit durch TLS

Da die LoversLamp über das Internet kommuniziert, müssen wir sicherstellen, dass die gesendeten Daten vor Angriffen geschützt sind. Ohne Verschlüsselung könnten Dritte die MQTT-Nachrichten abfangen und möglicherweise sogar manipulieren. Hier kommt TLS (Transport Layer Security) ins Spiel, ein Standardprotokoll zur Verschlüsselung von Internetkommunikation.

TLS stellt sicher, dass die Datenübertragung zwischen dem ESP32 und dem MQTT-Broker verschlüsselt und sicher ist. Verbindet man sich mit einem Server mittels TLS, erfolgt als Erstes eine bestimmte Abfolge von Aktionen. Diese Aktionen bilden den sogenannten TLS-Handshake:

1. Client Hello: Der ESP32 schickt eine Nachricht an den Server, in der er angibt, welche Verschlüsselungsmethoden er unterstützt. Außerdem wird eine zufällige Zahl generiert, die später für die Verschlüsselung genutzt wird.
2. Server Hello: Der MQTT-Server antwortet mit seiner eigenen Liste unterstützter Verschlüsselungsmethoden und sendet sein SSL-Zertifikat, das von einer vertrauenswürdigen Zertifizierungsstelle (CA) signiert wurde.
3. Zertifikatsprüfung: Der ESP32 überprüft, ob das Zertifikat des Servers gültig ist, indem

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover



Die Verkabelung der LoversLamp ist sehr einfach.

er es mit einer gespeicherten Liste vertrauenswürdiger Zertifikate vergleicht. Falls das Zertifikat nicht stimmt, wird die Verbindung abgelehnt.

4. Schlüsselaustausch: Der ESP32 und der Server erzeugen nun mit mathematischen Verfahren einen gemeinsamen geheimen Schlüssel aus dem Public Key des Servers. Dieser Schlüssel wird für die spätere Datenverschlüsselung genutzt.
5. Datenverschlüsselung beginnt: Ab diesem Punkt werden alle Daten verschlüsselt übertragen, sodass niemand die Nachrichten mitlesen oder verändern kann.

Ohne TLS könnten Angreifer in öffentlichen oder unsicheren Netzwerken die Datenpakete der LoversLamp mitlesen und sogar manipulieren. Ein Hacker könnte beispielsweise falsche MQTT-Nachrichten senden und die Lampen nach Belieben ein- oder ausschalten. Mit TLS ist die gesamte Kommunikation verschlüsselt und

Wichtige Stellen im Code

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <Adafruit_NeoPixel.h>


// Anzahl der LEDs und Pin definieren
#define LED_PIN 2       // GPIO-Pin für Datenleitung
#define NUM_LEDS 300      // Anzahl der LEDs im Streifen


Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB +
NEO_KHZ800);


// WLAN-Zugangsdaten
const char* ssid = "xxx";
const char* password = "xxx";


// MQTT HiveMQ Broker (Public Broker von HiveMQ oder
eigener Server)
const char* mqtt_server = "xxx";  // Ändere auf den
eigenen HiveMQ-Broker.
const int mqtt_port = 8883;  // TLS-Port für MQTT


// MQTT-Zugangsdaten
const char* mqtt_user = "esp32";
const char* mqtt_password = "xxx";
```

```
// MQTT-Topic

const char* mqtt_topic = "esp32/test";


// TLS-Zertifikat (Root CA für HiveMQ oder eigenen
Server)
const char* root_ca = R"EOF(
-----BEGIN CERTIFICATE-----
xxx
-----END CERTIFICATE-----


)EOF";


// WiFi- und MQTT-Clients erstellen
WiFiClientSecure espClient;
PubSubClient client(espClient);
```

In den Zeilen 13 und 14 wird die WLAN-Verbindung eingerichtet mit dem Netzwerknamen und dem Passwort. Beides muss vor dem Flashen auf den ESP geändert werden.

In den Zeilen 17 bis 25 sind die Daten für MQTT hinterlegt; sowohl die URL für den Server und dessen Port als auch die Login-Daten für den ESP-Client und die Topic, zu der Daten gesendet und empfangen werden sollen.

Zeile 28 bis 32: Im richtigen Code befindet sich hier das Root-Zertifikat, das bei der TLS-Verbindung als Referenz verwendet wird. Das muss das Zertifikat von HiveMQ sein. Den Link dazu findet man in der Kurzinfo.

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

controlWLED

```
void controlWLED() {
    if (ledState) {
        for (int i = 0; i < NUM_LEDS; i++) {
            strip.setPixelColor(i, strip.Color(255, 0, 0)); // Rot setzen
            }
            strip.show();           // Farben anzeigen
    } else {
        for (int i = 0; i < NUM_LEDS; i++) {
            strip.setPixelColor(i, strip.Color(0, 0, 0));
          }
          strip.show();
    }
}
```

In Zeile 4 dieses Codes kann man die Farbe der LEDs im eingeschalteten Zustand ändern. Aktuell sind sie auf Rot eingestellt. Über ein Webtool kann man sich die **R**(ed)**G**(reen)**B**(lue)-Werte für verschiedene Farben erzeugen lassen. Der Link dazu befindet sich in der Kurzinfo.

abgesichert, sodass nur der ESP32 und der HiveMQ-Server die Nachrichten entschlüsseln können.

Wie richten wir TLS für die LoversLamp ein?

Um TLS mit dem ESP32 zu nutzen, benötigen wir das Root-Zertifikat des MQTT-Servers. Dieses Zertifikat dient dazu, den Server zu identifizieren und sicherzustellen, dass wir uns mit dem richtigen Broker verbinden. Daneben steckt in dem Zertifikat der öffentliche Schlüssel, mit dem später der Schlüsselaustausch für TLS geschützt wird.

HiveMQ stellt ein öffentliches Root-Zertifikat zur Verfügung, das wir in unseren Code einbinden müssen (Link dazu in der Kurzinfo). Der ESP32 nutzt dann WiFiClientSecure, um eine verschlüsselte Verbindung zum MQTT-Server herzustellen. Wieso WiFiClientSecure benötigt wird, steht im Kasten „Was ist WiFiClientSecure?“.

Touch Me – Wie funktioniert der Touchsensor?

Das Sensorkonzept, das hier verwendet wird, nennt man kapazitiven Touch. Das ist die gleiche Technologie, die auch bei Smartphones für den Bildschirm verwendet wird.

Der ESP32 behandelt einen seiner Pins wie einen Kondensator und misst Schwankungen in der Kapazität (die Fähigkeit, elektrische Ladung zu speichern) dieses Kondensators. Berührt man den Pin, ändert sich dieser Kapazitätswert. Diese Änderung erkennt der ESP32 und kann daraufhin eine Aktion ausführen.

Hardware

Die Basis des Projektes bildet ein ESP32. Dieser steuert als Leuchtmittel einen WS2812-LED-Streifen. Um die Lampe zu bedienen, wird kein Taster verwendet, sondern ein Touchpin des ESP32 – genauer gesagt Pin G0. Details dazu stehen im Kasten „Touch Me – Wie funktioniert der Touchsensor“.

Verkabelt werden die LEDs wie in der Grafik zu sehen. Versorgt wird der Streifen direkt mit 5 Volt über den ESP. Der „+5V“-Anschluss des LED-Streifens kommt an einen 5-Volt-Pin des ESPs und der GND-Anschluss wird mit dem GND des ESP verbunden. Der Datenanschluss „Din“ kommt an Pin G2.

Wir bauen die Lampen

Die oben beschriebene Technik kann jetzt auf verschiedene Art und Weise in eine Lampe umgebaut werden. Zum Beispiel kann man die LED-Kette direkt an die Wand kleben oder an ein Möbelstück.

Für diesen Artikel wird die LoversLamp als Tischlampe umgesetzt. Diese Lampe besteht aus einem Holzfuß, einem Lampenschirm und einem Plastikrohr.

Als Erstes wird der Holzfuß vorbereitet. Dafür wird eine funktionierende Lampe auseinandergenommen. Diese Spenderlampe (Typ HITO-Tischlampe, verlinkt in der Kurzinfo) besteht aus dem Holzblock, der zum Fuß für die neue Lampe wird, einer eingelassenen 27er-Lampenfassung und einem Stromkabel. Die Lampenfassung wird zuerst ausgebaut. Dafür wird an der Unterseite des Holzblocks das weiche, rutschfeste Material vorsichtig abgezogen. Darunter befindet sich eine Vertiefung, in der das Stromkabel in die verbaute Lampenfassung übergeht. Die Lampenfassung ist mit einer Mutter festgeschraubt.

Jetzt schraubt man die Zugentlastung des Kabels (Kreuzschraube) ab, schneidet das Kabel nahe an der Mutter ab und zieht es aus dem Sockel heraus. Dann dreht man die Mutter mit einer Rohrzange von der Hohlachse herunter.

Jetzt kann man die 27er-Lampenfassung oben aus dem Sockel herausziehen. Damit ist der Sockel von aller alten Technik befreit.

Unter dem Gummi verbirgt sich eine kleine Einbuchtung.

Der ESP32 ist zwar schon ein kleines Board, aber trotzdem noch zu groß für das Sockelloch.

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

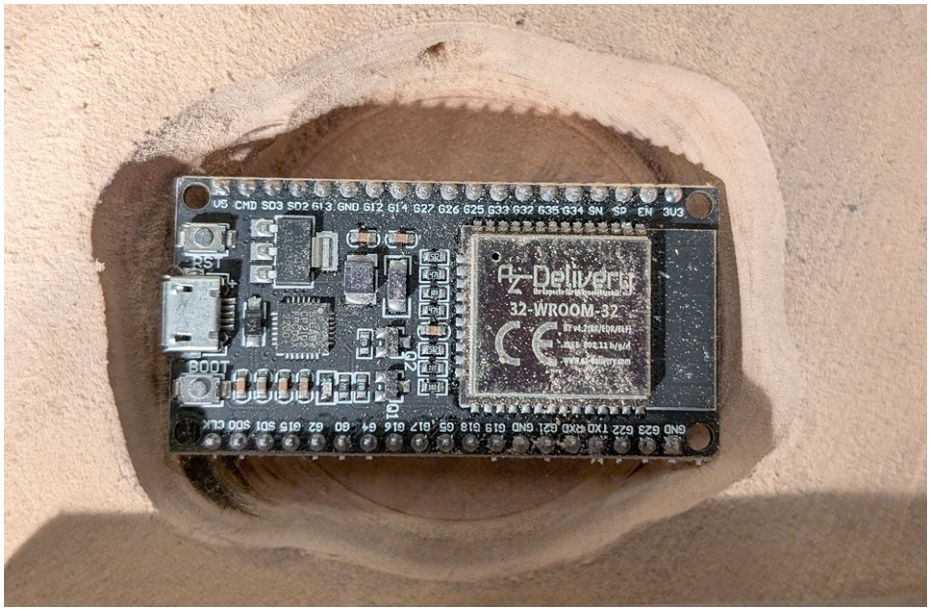
Was ist WifiClient-Secure?

WiFiClientSecure ist eine Erweiterung von WiFiClient, die TLS/SSL-Verschlüsselung unterstützt. Das bedeutet, dass die Verbindung zu einem Server über HTTPS (Port 443) erfolgen kann. Dadurch werden die Daten verschlüsselt übertragen, was wichtig ist, wenn sicherheitskritische Informationen, z. B. Passwörter oder API-Schlüssel, über das Internet gesendet werden.

WiFiClient ist für die Nutzung in einem LAN-Netzwerk geeignet. Diese Library kann keine verschlüsselten Serververbindungen aufbauen. Die Funktionen für TLS/SSL, die WiFiClientSecure bietet, fehlen. Wenn man aber über das Internet Daten versendet, sind die Sicherheitsfeatures von WiFiClientSecure wichtig, um die übertragenen Daten zu schützen.

Im Fall der LoversLamp überprüft eine Methode von WiFiClientSecure das Zertifikat des MQTT-Brokers, indem es das Serverzertifikat mit einem im Programm eingefügten vergleicht. Dadurch wird sichergestellt, dass das Programm mit dem richtigen Server verbunden ist. Dann kann die Übertragung der Daten, also der Zugangsdaten des Clients und der gesendeten Nachrichten, verschlüsselt geschehen.

Wie man in der Arduino IDE Bibliotheken installiert, wird in einem Artikel aus der Kurzinfo erklärt.



Mit einem spanenden Verfahren wird der Sockel ESP-kompatibel gemacht.

Die kleinen Klips an der Seite muss man vorsichtig aufhebeln. Sie dürfen nicht abbrechen.

Die Fassung brauchen wir noch. Sie muss aber angepasst werden.

In die nach unten zeigende Öffnung (in der man gerade die Mutter gelöst hat) kommt später der ESP32. Der ist allerdings etwas zu groß. Deshalb muss die Öffnung mit einem Dremel und einem 80er-Schleifpapier-Aufsatz vergrößert werden.

Wer das Projekt mit einem Wemos-D1-Mini nutzt, hat direkt viel Beinfreiheit im Sockel.

Sobald der ESP32 genug Platz im Sockel hat, kann man mit der Technik loslegen.

Das erwähnte Plastikrohr wird zum Hals der Lampe, um den man auch den LED-Streifen wickelt. Man beginnt damit, den LED-Streifen an einer Seite des Rohrs (im Weiteren „oben" genannt) eng herumzuwickeln.

Der LED-Streifen ist mit doppelseitigem Klebeband versehen. Damit der Streifen am Rohr hält, muss man nur den blauen Schutzfilm abziehen. Man umwickelt das Rohr bis zur Hälfte und schneidet überschüssige LEDs einfach mit einer Schere ab.

Am oberen Ende des Rohres hängen jetzt die Kabel, die für die Strom- und Datenverbindung als LEDs dienen. An diese Kabel lötet man jeweils 30 cm Klingeldraht an und schiebt die drei langen Kabel durch das Rohr. Zusätzlich führt man noch ein viertes Kabel in das Rohr. Dieses ist später für den Touchpin gedacht.

Nun legt man das Rohr erst mal beiseite und schnappt sich die Lampenfassung, die man vorhin aus dem Sockel ausgebaut hat. Diese Fassung hebelt man an den Seiten mit einem Schraubenzieher auf.

Dadurch erhält man zwei Teile: zum einen den Teil mit dem Gewinde und zum anderen den Teil mit der Metallstange, mit der die ganze Fassung am Sockel befestigt war.

An der Seite mit dem Gewinde muss man jetzt ein Loch durchbohren. Da kommen nämlich später wieder Kabel durch.

Jetzt fädelt man alle Teile wie bei einer Kette auf: die an den LED-Streifen angelöteten

Die Fassung passt genau auf das Rohr.

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

Kabel durch das Rohr, dann durch die durchbohrte Fassung (das Gewinde muss zum Rohr zeigen), dann durch den zweiten Teil der Fassung durch das Metallrohr mit dem Gewinde und als Letztes durch den Holzsockel.

Sind alle Kabel richtig durchgefädelt, drückt man die zweigeteilte Lampenfassung wieder zusammen und schraubt sie wieder in den Holzsockel.

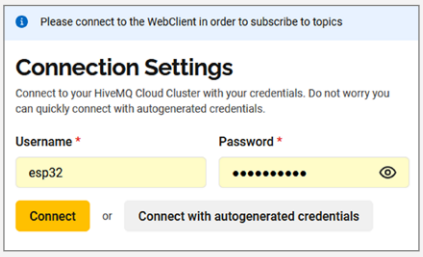
Ist das geschehen, drückt man das Rohr mit den LEDs in die Lampenfassung. Hierbei muss man ein bisschen sanfte Gewalt anwenden. Hält das Rohr nicht, kann man auch etwas Heißkleber zu Hilfe nehmen.

Auf der Unterseite des Sockels verkabelt man den ESP32, legt ein Stromkabel durch das Loch, durch welches das Stromkabel der alten Lampe ging, und klebt das rutschfeste Material wieder fest.

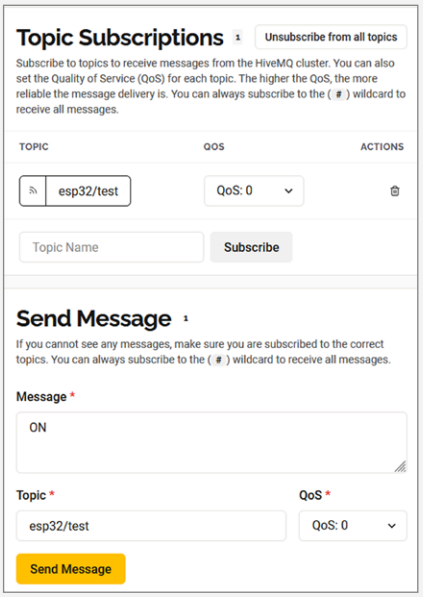
Jetzt ist der Lampenschirm dran. Für den gibt es in der Kurzinfo verlinkt eine Halterung

Toller Test-Tipp

Wenn man testen möchte, ob die Lampe richtig funktioniert und wirklich bei einem geänderten Zustand (der über MQTT empfangen wird) umschaltet, kann man das über HiveMQs Webclient probieren. In der HiveMQ-Übersicht klickt man dafür oben in der Leiste auf „Web Client". Dort meldet man sich dann zuerst mit den oben angelegten Zugangsdaten an und kann dann Nachrichten unter einer Topic veröffentlichen. Die Topic im Code dieses Projektes ist „esp32/test".



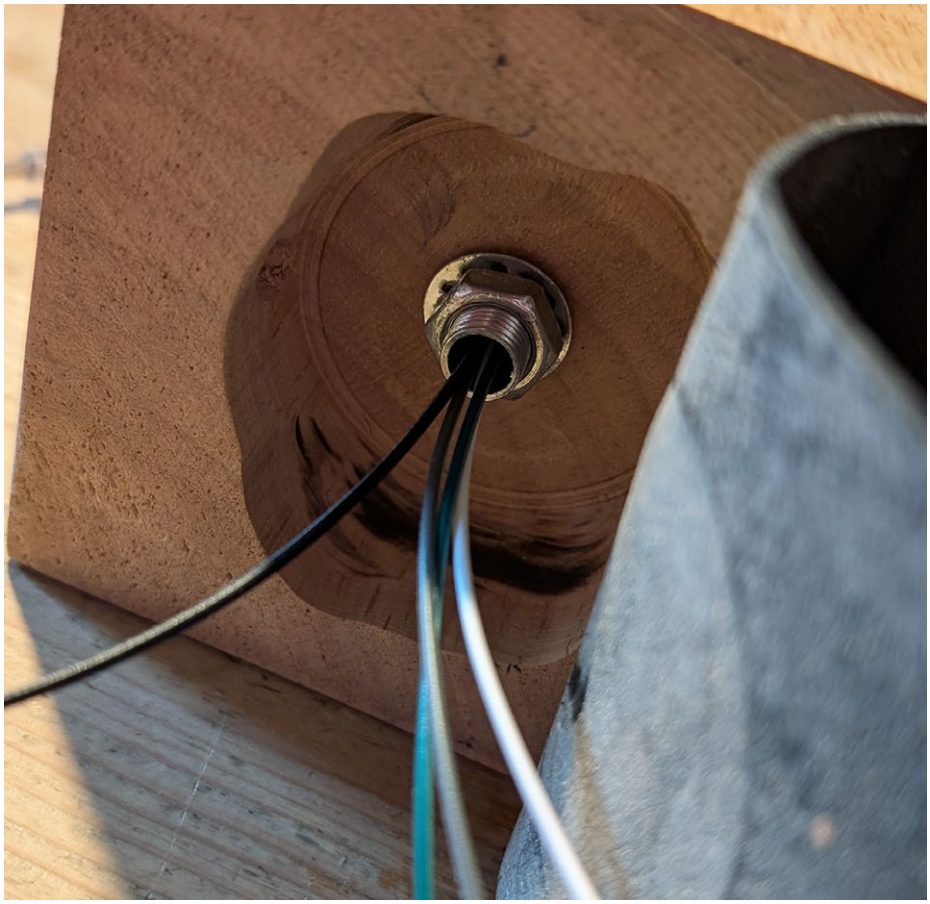
Unter „Connection Settings" meldet man den Webclient mit den Anmeldedaten an, die man festgelegt hat.



Danach kann man unter einer bestimmten Topic Nachrichten senden.

Die Halterung ist im Endeffekt ein flacher Teller.

Das kleine Kabelrohr an der Lampenfassung ist groß genug für alle Kabel.



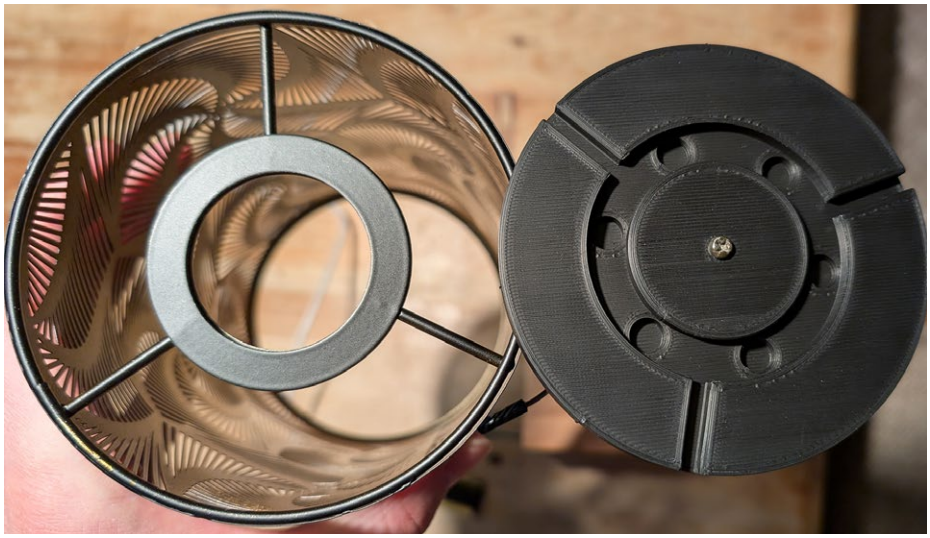
Und so enden alle Kabel in der erweiterten ESP-Kuhle.

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover

zum 3D-Drucken. Diese Halterung passt genau zu einem bestimmten Lampenschirm, der ebenfalls in der Kurzinfo verlinkt ist, und muss aus leitfähigem PLA gedruckt werden, damit man später die Oberseite der Lampe berühren kann, um sie anzuschalten.

In die Halterung klebt man zuerst ein paar kleine Magnete ein (siehe Bild). Sie halten später den Lampenschirm fest. Dann schraubt man auf der Unterseite der Halterung das lose Kabel für den Touchpin des ESPs mit einer Schraube so ein, dass es Kontakt mit dem Plastik der Halterung hat. Nun setzt man die Halterung auf das Rohr an der Oberseite auf und legt den Lampenschirm auf die Magnete.

Jetzt ist die LoversLamp fertig. Wer noch niemanden für eine zweite Lampe hat, wird mit seinen Maker-Fähigkeiten bestimmt bald jemanden finden! *—das*



Die Halterung sitzt ganz oben auf dem Rohr.

1/2 quer Rechts

© Copyright by Maker Media GmbH.
Persönliches PDF für alle Leser der Make aus 30625 Hannover